

## PLAYGROUND · LESSON 1

# The Sigil Playground

A phrase becomes a path of light — encode a word on the Square of Mars and draw it as a glowing sigil on a 64×64 LED wall.

---

PLAY-001 · ~60 MIN · INTERMEDIATE

---

*This is a creative build, framed plainly as that. There is no EML guard and no safety contract here — nothing to clamp, nothing to prove. The charm of the project is that it is honest fun. Keep it that way.*

## 1. The one idea

A phrase becomes a path of light. You map a word onto a 5×5 number grid (the **Square of Mars**), trace the path that connects its letters in order, and draw that path as a glowing **sigil** on a 64×64 LED wall.

## 2. Why it matters

This is an encoding craft. The same move — take meaning, turn it into a rule, turn the rule into something you can see — is the heart of every generative-art and data-visualization project. Here the output is beautiful instead of useful, and that is the whole point: you are practising the make-it-legible-and-lovely skill on a canvas where the only judge is your own eye.

## 3. Parts & wiring

- ESP32 dev board
- 64×64 RGB LED matrix panel (HUB75 interface, 1/32 scan)
- A HUB75 driver/adapter for the ESP32 (e.g. an ESP32-Trinity-style board or a level-shifter adapter) — **strongly recommended**; it maps the 13 HUB75 control lines to the ESP32 and shifts 3.3V logic up to 5V
- A 5V power supply that can deliver **several amps** — a full-white 64×64 panel can pull 3–4A; never power the panel from the ESP32
- The **ESP32-HUB75-MatrixPanel-I2S-DMA** library (Arduino Library Manager)
- Optional: a small passive buzzer or speaker for the percussion in §9

HUB75 is a 16-pin ribbon: three RGB pairs (upper/lower half), five address lines (A–E), plus CLK, LAT, and OE. You do **not** wire these one by one from memory — the adapter board fixes the mapping, and the library holds the pin numbers in its config. The job here is to match the library's pin config to your adapter, not to invent a pinout.

*One real constraint to design around: HUB75 panels are demanding to drive on an ESP32. The refresh is CPU-heavy and the timing is fussy. The matrix library does the heavy lifting over I2S + DMA, but it shapes your architecture — let it.*

## 4. Predict (do this before uploading)

Write down your guesses:

- Your phrase reduces to a list of grid cells (1–25). Trace it by hand on a 5×5 square first. Where does the path start? Where does it end?
- On the 64×64 wall, each 5×5 cell becomes a ~12-pixel block. How wide will one node look? Will the whole sigil fill the wall or float in the middle?
- Full brightness on every LED would draw ~3–4A. At the dim colours you'll draw, guess the current — more or less than the ESP32's USB port can give?

Hold those guesses. §6 checks them.

## 5. Build it — the smallest version that works

The smallest version is: **light the path**. No fades, no pulses yet — just the nodes and the lines of one sigil, drawn once. Beauty comes in §9.

Encode the phrase by hand first (this lesson keeps the encoding off-device so the one idea stays "draw the path", not "parse text"):

1. Number the 5×5 Square of Mars cells 1–25, row by row.
2. Convert each letter of your phrase to a number 1–25 (A=1 ... Z=26, then subtract 25 from anything over 25), and drop repeats that don't move you.

1. The ordered list of cells is your path. Mark the first cell with a small circle and the last with a short bar — the traditional start/end of a sigil.

Then draw it:

```

#include <ESP32-HUB75-MatrixPanel-I2S-DMA.h>

MatrixPanel_I2S_DMA *wall = nullptr;

const int N = 64;           // panel is 64 x 64
const int CELL = N / 5;     // ~12 px per Square-of-Mars cell

// Centre of grid cell n (1..25) in panel pixel coordinates.
void cellCenter(int n, int &x, int &y) {
    int idx = n - 1;
    x = (idx % 5) * CELL + CELL / 2;
    y = (idx / 5) * CELL + CELL / 2;
}

// Your phrase, already reduced to a path of cells (1..25):
const int sigil[] = { 8, 14, 2, 20, 11, 17 };
const int sigillen = sizeof(sigil) / sizeof(sigil[0]);

void setup() {
    HUB75_I2S_CFG cfg(64, 64, 1);    // width, height, one chained panel
    wall = new MatrixPanel_I2S_DMA(cfg);
    wall->begin();
    wall->setBrightness8(90);          // dim - kind to your eyes and your PSU
    wall->clearScreen();

    uint16_t glow = wall->color565(120, 90, 255);    // soft violet

    // Draw the connecting lines, then the nodes on top.
    for (int i = 0; i < sigillen - 1; i++) {
        int x1, y1, x2, y2;
        cellCenter(sigil[i], x1, y1);
        cellCenter(sigil[i + 1], x2, y2);
        wall->drawLine(x1, y1, x2, y2, glow);
    }
    for (int i = 0; i < sigillen; i++) {
        int x, y;
        cellCenter(sigil[i], x, y);
        wall->fillCircle(x, y, 3, glow);
    }
}

```

```

    }
}

void loop() {
    // Static sigil for now. Animation lives in §9.
}

```

Naming the shape of it: **map** → **place** → **draw**. Encode the phrase to cells, place each cell on the wall, draw lines and nodes. That is the whole lesson.

## 6. Test against your prediction

Power the panel from its 5V supply, the ESP32 from USB, and upload. The sigil should appear floating on the wall. Did the path start and end where you traced it by hand? Is each node about 12 pixels across, as you guessed? Hold a phone camera up — at this brightness the LEDs read as a soft glow, not a glare. That gap between "full white = blinding 3–4A" and "what you actually drew" is the prediction worth feeling.

## 7. What goes wrong (pre-loaded debugging)

- **Panel stays dark** → it is almost never the code. Check the 5V supply and that

the panel has its own power; an ESP32 cannot light a 64×64 wall. Check the ribbon orientation and that the library's pin config matches your adapter.

- **Top and bottom halves wrong, or the image is doubled** → the scan rate is

misconfigured. A 64-high panel is 1/32 scan and needs the **E** address line; if E isn't mapped you get two stacked half-images.

- **Colours swapped (red shows as blue)** → the RGB channel order in the library

config doesn't match the panel. Flip it in the config, not in your colours.

- **Flicker or tearing** → brightness too high, or the DMA refresh is starved.

Lower `setBrightness8`, and don't block in `loop()`.

- **ESP32 browns out / reboots** → you tried to feed panel current through the

board. Separate the rails; share only ground.

## 8. Checkpoint

You drove a 64×64 HUB75 LED wall from an ESP32 and turned a phrase into a drawn sigil — meaning encoded as a path, the path rendered as light. You now own the loop **encode** → **place** → **draw**, which is every generative-display project in miniature.

## 9. Push further

This is where the build gets beautiful. Add them one at a time:

- **Anti-aliased lines** so the connections draw smoothly instead of as stair-steps.
- **A brightness pulse** on each node as its beat plays — drive a passive buzzer on

a spare GPIO so the sigil materialises *to* a rhythm.

- **Fade trails** behind the drawing point, so the sigil grows rather than appears.
- **On-device encoding:** move the letter→cell reduction onto the ESP32 so you can

type a new phrase and watch a new sigil draw itself. That is its own lesson — keep it separate, and keep this one about the light.